

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA1407

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

***CO4:** Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 10%

QUESTIONS:

Total Marks:50

Annexure A

DATA INTERPRETATION

Code of Conduct:

I Rekha S K (192511023) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Rekha S K

Annexure A

DATA INTERPRETATION

Q1. Intermediate Code Analysis and Optimization

Given the three-address code:

$t1 = x + y$ $t2$

$= t1 * z$ $t3 =$

$t2 / w$ $t4 =$

$t3 - p$

Analyze the sequence of operations and construct the equivalent expression tree. Evaluate how temporary variables improve intermediate code generation and reduce repeated computations.

Q2. Symbol Table and Memory Allocation

Given the following symbol table entries:

Variable Data Type Size (Bytes)

p char 1 q double 8 r int

4 s float 4

Interpret how the compiler allocates memory for these variables and calculate the total memory required. Analyze how alignment and data types affect memory organization.

Q3. Optimization in Intermediate Code

Given the intermediate code:

$t1 = m * n$

$t2 = p + q$

$t3 = m * n$

$t4 = t2 - t3$

Trace the execution step-by-step and identify optimization opportunities such as common subexpression elimination and redundant computation removal. Explain how optimization improves execution efficiency.

Q4. Control Flow and Boolean Expression Handling

Given the Boolean expression: $\text{if } (x > y \parallel p < q)$

Intermediate code: $\text{if } x > y \text{ goto L1 if } p < q \text{ goto L1}$

goto L2

L1: ...

Interpret the control flow of the given intermediate representation and explain how short-circuit evaluation minimizes unnecessary condition checking during execution.

Q5. Backpatching and Flow Control Given:

Instruction	Target
-------------	--------

200: if x==y goto _	
---------------------	--

201: goto _	
-------------	--

Backpatch lists:

- truelist = {200}
- falselist = {201}

Analyze the role of backpatching in intermediate code generation. Demonstrate how target addresses are updated during control flow handling and explain its importance in Boolean expression translation.

Answers

Q1. Intermediate Code Analysis and Optimization Given three-address code:

t1 = x + y
t2 = t1 * z
t3 = t2 / w
t4 = t3 - p

Data Interpretation

Each line of the three-address code (TAC) contains exactly one operator on the right-hand side, so it maps directly onto one internal node of an expression tree. Reading the sequence top to bottom shows a chain of dependency: t1 feeds into t2, t2 feeds into t3, and t3 feeds into t4. This means the code was generated by a post-order (bottom-up) traversal of the source expression

$((x + y) \times z) / w - p$, which is exactly what t4 evaluates to.

Expression Tree for t4 = ((x + y) * z / w) - p

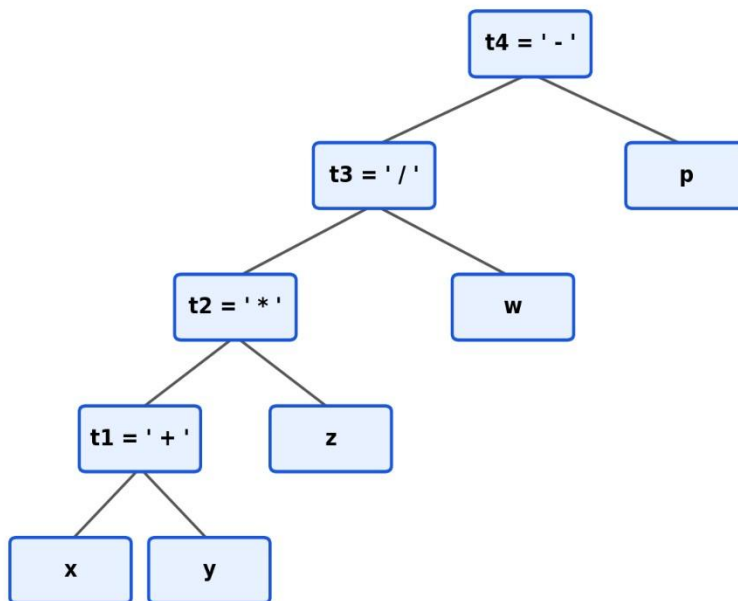


Fig. 1 — Expression tree reconstructed from the TAC (built bottom-up: leaves are source operands, each internal node is one TAC instruction).

Analysis

- Each temporary (t1–t4) corresponds to one internal node of the tree, and the operator stored with that temporary is the label of the node.

- The tree is left-skewed because every operation immediately consumes the previous temporary (t1 in t2, t2 in t3, t3 in t4) — this reflects the left-to-right associativity of +, ×, /, − used in the source expression.
- Leaves x, y, z, w, p are never recomputed; each appears exactly once, so no operand is fetched from memory more than necessary.

Application of Concepts — Role of Temporary Variables

- Linearisation: temporaries convert a nested, tree-shaped expression into a flat sequence of simple instructions, which is the form a code generator or virtual machine can execute directly.
- Single evaluation of sub-results: once $t1 = x + y$ is computed, its value is reused by name instead of re-evaluating $(x + y)$ wherever it is needed — this is the same principle that common-subexpression elimination (Q3) exploits.
- Register/name allocation: a temporary is a symbolic register; the compiler's register allocator can later map t1..t4 onto physical registers and free a register as soon as its temporary is no longer live, reducing memory traffic.
- Simplifies optimization passes: because each instruction has at most one operator, later optimizations (constant folding, CSE, dead-code elimination) can be applied instruction-by-instruction rather than by re-parsing an expression tree each time.

Conclusion

The TAC sequence is a faithful, one-operator-per-line linearisation of the expression tree in Fig. 1. Temporary variables act as named placeholders for intermediate sub-results, which (a) eliminates re-computation of any sub-expression, (b) exposes clear points where optimizations such as CSE and dead-code elimination can be applied, and (c) produces code that is straightforward to translate into target-machine instructions or to map onto a fixed set of registers.

Q2. Symbol Table and Memory Allocation

Given symbol table entries:

Variable	Data Type	Size (Bytes)	Alignment Req.	Offset (start–end)
p	char	1	1-byte	0 – 0
(padding)	—	7	—	1 – 7
q	double	8	8-byte	8 – 15
r	int	4	4-byte	16 – 19
s	float	4	4-byte	20 – 23

Data Interpretation

The symbol table records, for every declared identifier, its data type and the storage size the compiler must reserve for it. Summing the raw sizes gives the minimum data actually stored:

$1(p) + 8(q) + 4(r) + 4(s) = 17$ bytes. However, the compiler does not simply place these 17 bytes back-to-back — it must also satisfy the target architecture's alignment rules.

Analysis — Effect of Alignment

- On a typical 64-bit architecture, a variable of size n bytes must start at an address that is a multiple of n (natural alignment): $\text{char} \rightarrow 1\text{-byte boundary}$, $\text{int/float} \rightarrow 4\text{-byte boundary}$, $\text{double} \rightarrow 8\text{-byte boundary}$.
- Allocating in declaration order (p, q, r, s): p occupies offset 0. The next variable, q (double), needs an 8-byte-aligned address, but offset 1 is not a multiple of 8, so the compiler inserts 7 bytes of padding before q . r and s then fall naturally on 4-byte boundaries with no further padding needed.
- Total allocated memory = 17 bytes of actual data + 7 bytes of padding = 24 bytes (also the smallest multiple of $8 \geq 17$, since the block's overall alignment is dictated by its largest member, the double).
- If the compiler is free to reorder fields (as some compilers do for structures) — largest to smallest: q, r, s, p — the same 7 bytes of padding still appear, but only as trailing padding after p rather than between p and q . The total size remains 24 bytes; only the layout and location of wasted space changes.

Application of Concepts

- Symbol table role: each entry supplies (name, type, size) which the storage-allocation phase of the compiler consumes to compute base offsets for the activation record or global data segment.
- Offset computation: $\text{offset}(\text{next}) = \text{align_up}(\text{offset}(\text{prev}) + \text{size}(\text{prev}), \text{alignment}(\text{next}))$ — this is exactly the rule applied above to place q at offset 8.
- Trade-off exposed: declaration order affects internal fragmentation; ordering variables from largest to smallest alignment requirement is a simple compiler/programmer optimization that minimizes wasted padding.

Conclusion

Raw data requires only 17 bytes, but alignment constraints force the compiler to allocate 24 bytes — a 41% memory overhead from padding alone. This shows that memory organization in a compiler is governed not just by the sum of variable sizes but by the interaction between declaration order and each type's alignment requirement, which the symbol table must capture to generate correct and efficient storage layouts.

Q3. Optimization in Intermediate Code

Given intermediate code:

```
t1 = m * n
t2 = p + q
t3 = m * n
t4 = t2 - t3
```

Data Interpretation — Step-by-Step Trace

- **Step 1:** $t1 = m * n \rightarrow$ computes the product of m and n and stores it in $t1$.
- **Step 2:** $t2 = p + q \rightarrow$ an independent computation, stored in $t2$.

- **Step 3:** $t3 = m * n \rightarrow$ computes the exact same expression as Step 1 (m and n are unchanged in between), so $t3$ is guaranteed to equal $t1$.
- **Step 4:** $t4 = t2 - t3 \rightarrow$ uses the redundant result $t3$.

Analysis — Optimization Opportunities

- Common Sub-expression Elimination (CSE): $m * n$ is computed twice (Step 1 and Step 3) with no intervening assignment to m or n , so the second computation is redundant and $t3$ can simply be replaced by $t1$.
- Redundant computation removal: after CSE, the instruction $t3 = m * n$ itself is no longer needed — copy propagation lets Step 4 reference $t1$ directly, eliminating $t3$ entirely.
- Optimized code has only 3 instructions instead of 4, and one multiplication is removed:

```
t1 = m * n
t2 = p + q
t4 = t2 - t1
```

Application of Concepts

- CSE is typically detected using a DAG (directed acyclic graph) representation or a value-numbering table: identical operator + identical operand values $\Rightarrow$ identical node reused result. $\Rightarrow$
- This optimization is local (within a basic block) here, but the same principle extends globally across basic blocks using available-expressions data-flow analysis.
- Removing one multiplication reduces both instructions count and register pressure (one fewer live temporary, $t3$), which also shortens the critical path for instruction scheduling.

Conclusion

Tracing the code exposes that $t3$ is a pure duplicate of $t1$. Applying CSE followed by copy propagation removes one arithmetic operation and one temporary variable without changing the program's semantics. In loops or frequently executed code, eliminating even a single redundant multiplication like this can produce a measurable improvement in execution efficiency and reduced register/memory traffic.

Q4. Control Flow and Boolean Expression Handling

Given Boolean expression: $\text{if } (x > y \parallel p < q)$

Generated intermediate code:

```
if x > y goto L1
if p < q goto L1
goto L2
L1: ...
```

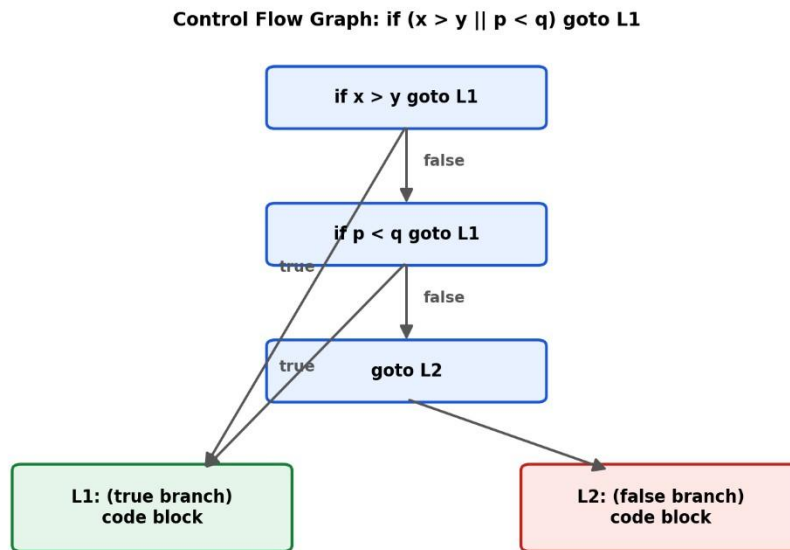


Fig. 2 — Control-flow graph for the OR (||) condition; the second test is skipped whenever the first is true.

Data Interpretation

The intermediate code implements logical OR without ever materializing a Boolean value in a variable — instead, control itself branches directly to L1 (true) or L2 (false). Each relational test is compiled into its own conditional jump, and the jumps are chained so that either test succeeding is sufficient to reach L1.

Analysis — How Short-Circuit Evaluation Works Here

- Instruction 1 evaluates $x > y$. If true, control jumps immediately to L1 — instruction 2 ($p < q$) is never reached, i.e. it is never evaluated.
- Only if $x > y$ is false does control fall through to instruction 2, which evaluates $p < q$. If this is true, control also jumps to L1; otherwise, control falls through to goto L2.
- This mirrors the semantics of || in languages like C/Java: the right operand of || is evaluated only when the left operand is false.

Application of Concepts

- This is the classic 'jumping code' translation scheme (Aho–Sethi–Ullman) for Boolean expressions, where a Boolean expression is translated using truelist/falselist of incomplete jumps rather than by computing a 0/1 value — this directly connects to backpatching in Q5.
- Efficiency gain: short-circuiting avoids evaluating unnecessary conditions, which matters when the second condition is computationally expensive (e.g., a function call) or has side effects.
- Safety benefit: short-circuiting also prevents evaluating conditions that could be invalid when the first condition already determines the outcome (e.g., guarding a pointer dereference with $p \neq \text{null} \parallel \dots$).

Conclusion

The control-flow translation of $x > y \parallel p < q$ performs at most one comparison when the first operand is true, and both comparisons only when the first is false. This confirms that short-circuit evaluation is implemented structurally through conditional branching in the intermediate code — not through explicit Boolean arithmetic — minimizing unnecessary condition checks and improving runtime efficiency.

Q5. Backpatching and Flow Control Given:

200: if x == y goto _

201: goto _

Backpatch lists:

- truelist = {200}
- falselist = {201}

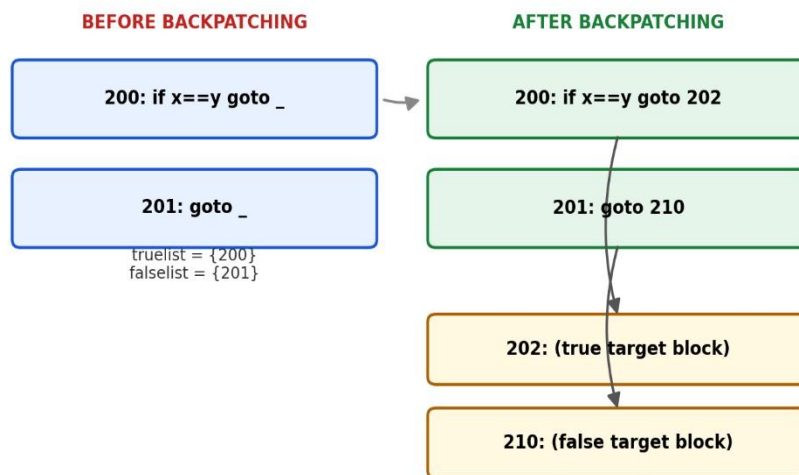


Fig. 3 — Instructions 200 and 201 hold placeholder targets until the true/false destinations are known, then `backpatch()` fills them in.

Data Interpretation

Instructions 200 and 201 are generated with an intentionally incomplete jump target ('_') because, at the point they are emitted, the compiler does not yet know where the true-branch code or the false-branch code will begin. Instead of leaving these jumps unresolved, the compiler records their instruction numbers in two lists: `truelist` (jumps to take when the condition is true) and `falselist` (jumps to take when it is false).

Analysis — How Targets Get Filled In

- The compiler continues generating code for whatever comes next in the program.
- As soon as the true-branch code's starting address becomes known (say, instruction 202), the compiler calls `backpatch(truelist, 202)`, which walks the `truelist` and overwrites the blank in every listed instruction — here, instruction 200 becomes 'if x == y goto 202'.

- Similarly, once the false-branch/next-statement address is known (say, instruction 210), `backpatch(falselist, 210)` rewrites instruction 201 to 'goto 210'.
- If a Boolean expression is built from sub-expressions (e.g., an `&&` or `||` of two comparisons), their `truelist/falselist` values are merged (via a `merge()` function) before the final `backpatch` call, so a single target fix-up updates every pending jump belonging to that sub-expression.

Application of Concepts

- Backpatching solves the classic forward-reference problem: a jump instruction is emitted before its target label exists in the code.
- It enables single-pass code generation — the compiler never needs to rescan or re-generate earlier code once a label's address is determined; it only needs to patch a numeric field in already-emitted instructions.
- This technique is the standard mechanism (Aho–Sethi–Ullman 'jumping code' scheme) used to translate `if`, `if-else`, `while`, and short-circuit Boolean expressions such as the one analyzed in Q4.

Conclusion

Backpatching allows a compiler to generate correct control-flow code in a single pass by deferring the resolution of jump targets: incomplete instructions are tracked in `truelist/falselist` and are patched retroactively the moment their true destination becomes known. This is essential for translating Boolean expressions and control constructs efficiently, without the overhead of multiple compilation passes or a separate back-fix stage.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided

Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand